

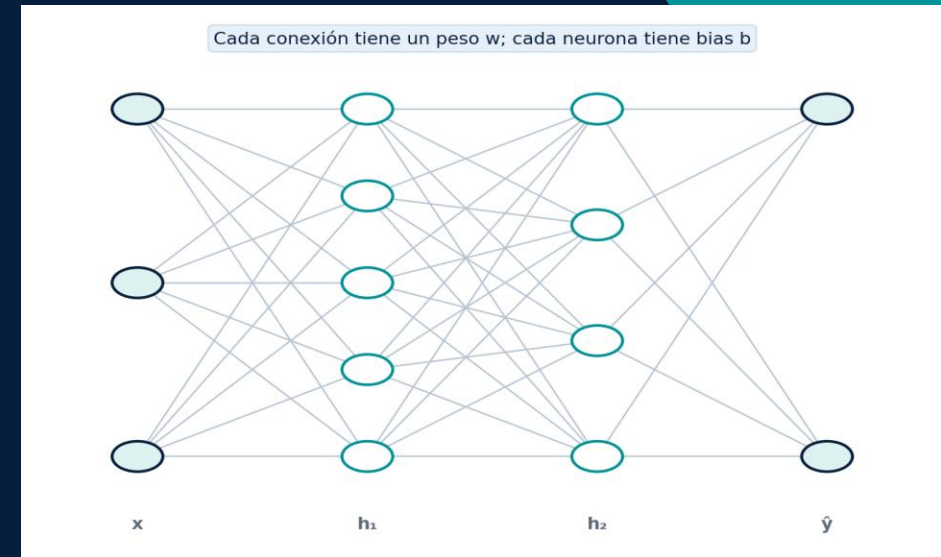
Redes neuronales multicapa y Deep Learning

Enfoque matemático, gráfico y explicativo

Materia: Inteligencia Artificial · 4º año · Lic. en Sistemas de Información

Jueves 21 de mayo de 2026

Profesores:
Mg. Díaz Santiago R.
AS. López Joaquín
BioIng. Hadad Alejandro



Objetivos de aprendizaje

Apertura

- Comprender una red neuronal multicapa como una composición de funciones parametrizadas.
- Explicar matemáticamente feedforward, gradiente descendente y backpropagation.
- Interpretar pesos, bias, capas, funciones de activación y pérdidas según el tipo de problema.
- Distinguir error de entrenamiento, validación y prueba; sesgo, varianza, underfitting y overfitting.
- Usar MSE, entropía cruzada, matriz de confusión, ROC/AUC y cross-validation para analizar desempeño.

Idea central

Una red profunda no “aprende reglas mágicas”: ajusta parámetros para reducir una función de pérdida sobre datos observados, intentando generalizar a datos no vistos.

Nivel esperado

La meta no es memorizar fórmulas, sino poder derivarlas y razonar qué cambia cuando se modifica la arquitectura, la pérdida o los datos.

Resultado práctico

Al finalizar, se podrá entrenar, diagnosticar y defender una MLP para regresión o clasificación con criterio matemático.

Mapa conceptual de la clase

Apertura



Lectura matemática

Una red con L capas transforma una entrada x mediante una cadena: cada capa aplica una transformación afín y una no linealidad. El aprendizaje ajusta todos los pesos y bias para minimizar una pérdida.

Red como composición

$$f_{\theta}(x) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(x)$$

θ contiene todos los pesos W y bias b .

¿Qué es una red neuronal multicapa?

Definición operativa

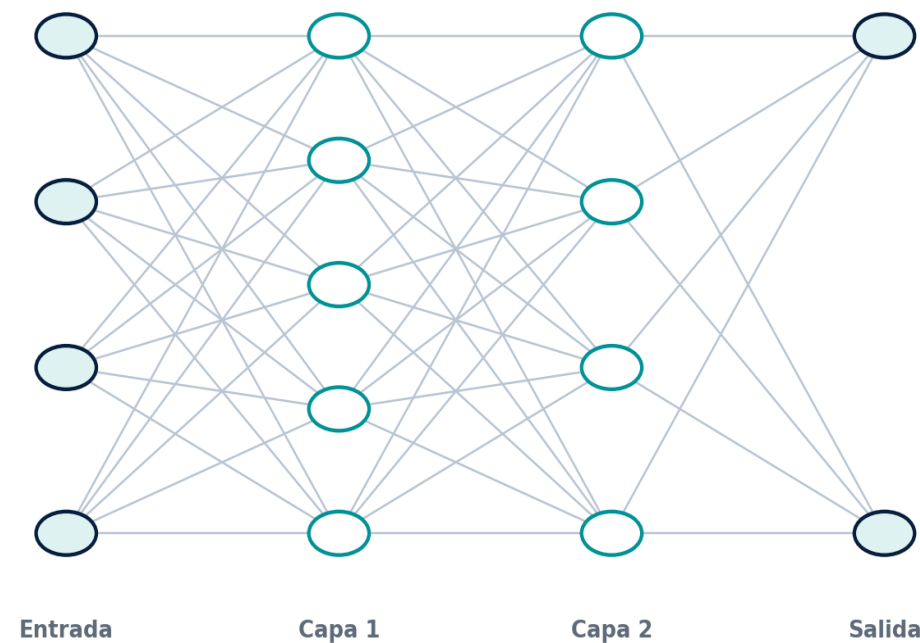
Una MLP transforma vectores de entrada en salidas mediante capas densas. Cada neurona combina entradas, agrega bias y aplica una activación.

Neurona

$$z = w^T x + b, \quad a = \phi(z)$$

- Pesos W : controlan la orientación e importancia de las señales.
- Bias b : permite umbrales/desplazamientos independientes de x .
- Activación ϕ : introduce no linealidad; sin ella, muchas capas equivalen a una sola transformación lineal.

Cada conexión tiene un peso w ; cada neurona tiene bias b



MLP = Multilayer Perceptron: una arquitectura feedforward clásica para regresión y clasificación.

Teorema: aproximación universal

Enunciado informal

Bajo condiciones amplias, una red feedforward con al menos una capa oculta, activación no lineal adecuada y suficientes neuronas puede aproximar cualquier función continua sobre un conjunto compacto con precisión arbitraria.

Forma matemática

$$\forall \varepsilon > 0, \exists f_{\theta} \text{ tal que } \sup_{x \in K} |f(x) - f_{\theta}(x)| < \varepsilon$$

K compacto; f continua; θ parámetros de la red.

Lo que NO garantiza

El teorema no dice que el entrenamiento encuentre esa red, ni que generalice, ni que use pocos datos. Aproximar no es lo mismo que aprender bien.

Interpretación para la materia

Las redes son familias muy flexibles de funciones. La pregunta práctica es: ¿qué arquitectura, pérdida, datos y regularización permiten aprender una aproximación útil?

Referencias clásicas: Cybenko (1989), Hornik (1991).

La neurona artificial: suma ponderada + bias + activación

Paso 1: combinación afín

$$z = \sum_{j=1}^d w_j x_j + b$$

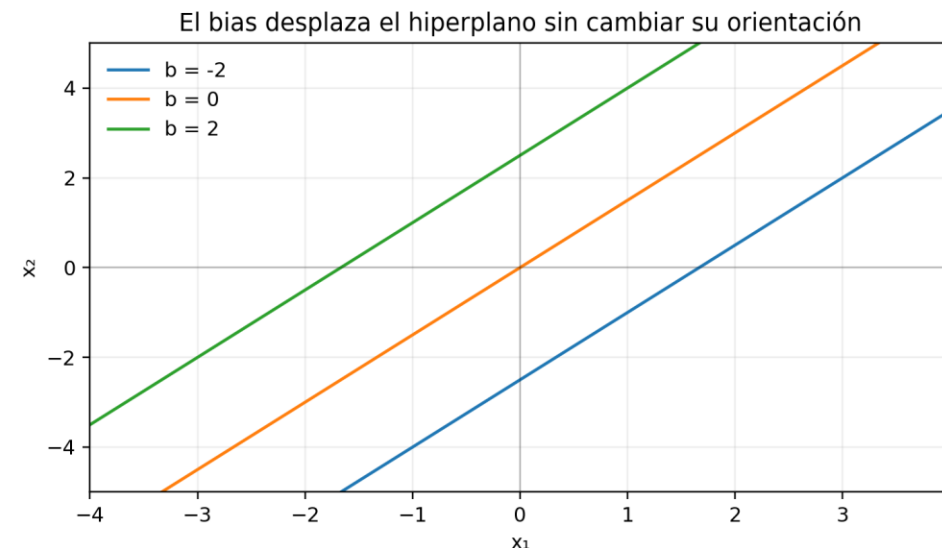
El bias b se suma aunque todas las entradas sean cero.

Paso 2: no linealidad

$$a = \phi(z)$$

Intuición geométrica

$z = 0$ define un hiperplano. Los pesos definen la orientación; el bias desplaza la frontera. En clasificación, esto modifica el umbral de decisión.



Ejemplo de interpretación

Si una neurona detecta “evidencia de fraude”, el bias representa el nivel base de sospecha. Un bias alto puede activar la neurona aun con señales débiles; uno bajo exige evidencia más fuerte.

¿Para qué sirve el bias?

1. Umbral flexible

Permite que la neurona se active a partir de cierto nivel de evidencia, no necesariamente cuando $w^T x = 0$.

2. Ajuste de intercepto

En regresión, cumple el rol de término independiente: mueve la función hacia arriba/abajo o la frontera en el espacio.

3. Representación más rica

Sin bias, muchas funciones simples no pueden representarse bien porque todas las fronteras quedan obligadas a pasar por el origen.

Modelo lineal sin bias

$$\hat{y} = w^T x$$

Modelo lineal con bias

$$\hat{y} = w^T x + b$$

Capa densa

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$$

Interpretación concreta

En un sistema que predice abandono universitario, el bias de una neurona puede capturar una tendencia base de riesgo antes de considerar variables como regularidad, asistencia o notas.

Capas, dimensiones y notación matricial

Forward por capa

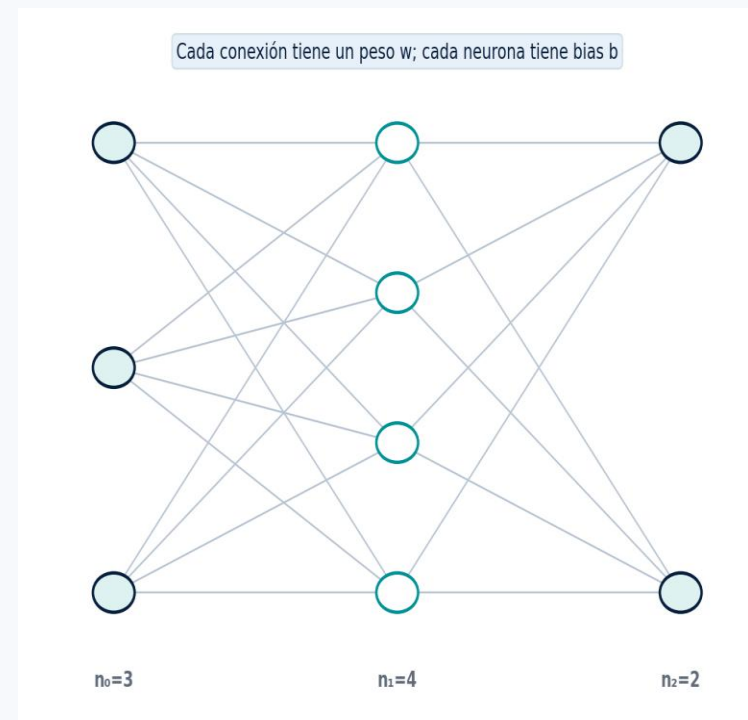
$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \phi^{(\ell)}(z^{(\ell)})$$

Dimensiones

Si la capa ℓ tiene n_ℓ neuronas y la anterior $n_{\ell-1}$, entonces $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $b^{(\ell)} \in \mathbb{R}^{n_\ell}$, $z^{(\ell)} \in \mathbb{R}^{n_\ell}$.

Por qué importa

La mayoría de los errores de implementación en redes densas son errores de forma: matrices mal orientadas, batch mal organizado o broadcasting no deseado.



Regla práctica: diseñar arquitectura = elegir dimensiones $n_0, n_1, \dots, n_L$ y activaciones.

Feedforward: de datos a predicción

Forward

- Entrada: vector x o mini-batch X .
- Cada capa calcula una transformación afín y una activación.
- La última capa produce la predicción $\hat{y}$: número, probabilidad o vector de probabilidades.
- El proceso es “hacia adelante”: no se ajustan pesos; solo se evalúa la función actual.



Para una muestra

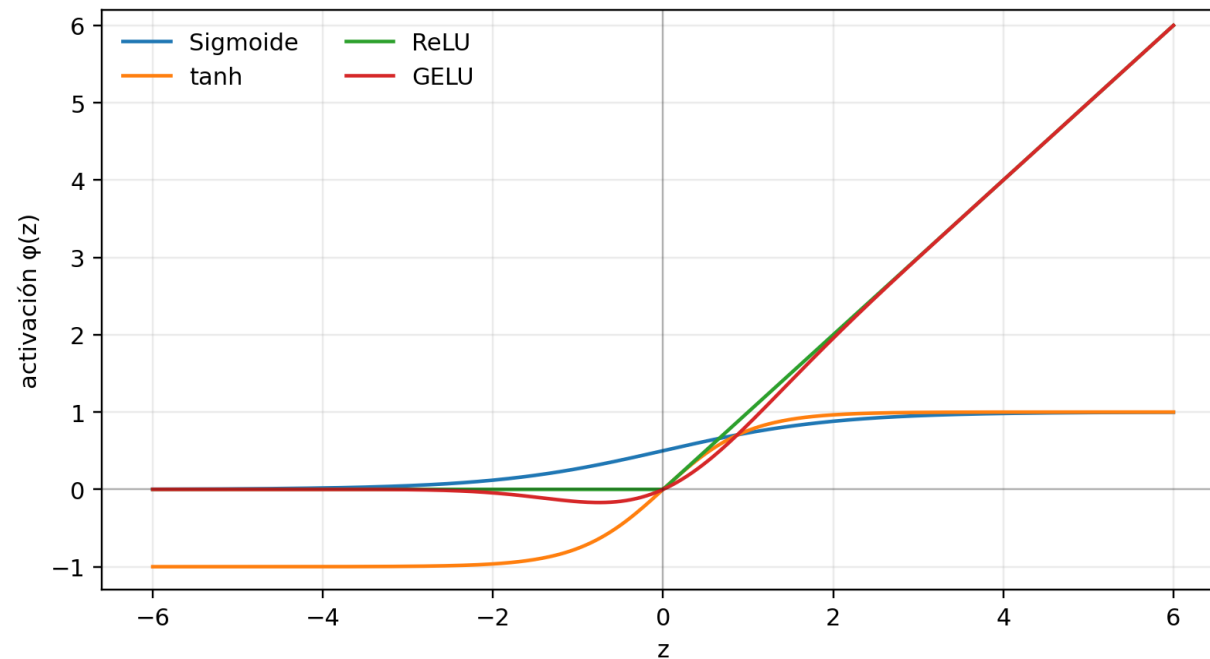
$$\hat{y} = f_{\theta}(x) = a^{(L)}$$

Pseudocódigo

```
a = x
para ℓ = 1,...,L:
    z = Wℓ a + bℓ
    a = φℓ(z)
retornar a
```

Funciones de activación usuales

Forward



ReLU

$\phi(z) = \max(0, z)$. Muy usada en capas ocultas; barata, no satura para z positivo, pero puede “morir” si queda siempre en 0.

Sigmoide / tanh

Útiles históricamente y para probabilidades binarias. Pueden saturar: derivadas cercanas a 0 dificultan el aprendizaje profundo.

GELU / Swish

Activaciones suaves usadas en arquitecturas modernas. Mejoran flujo de gradiente en algunos contextos, con más costo computacional.

Regla MIT-práctica: no elegir activación por costumbre; elegirla por estabilidad del gradiente, tipo de salida y escala de datos.

Activaciones de salida según el problema

Forward

Regresión

Salida lineal: $\hat{y} \in \mathbb{R}$. Ejemplo: predecir demanda energética, precio, tiempo de falla o concentración química.

Clasificación binaria

Sigmoide: devuelve una probabilidad $p(y=1 | x)$. Se decide clase con umbral, por ejemplo 0.5 o un umbral calibrado.

Multiclase

Softmax: transforma logits en probabilidades que suman 1. Útil cuando las clases son mutuamente excluyentes.

Regresión

$$\hat{y} = z^{(L)}$$

Binaria

$$p = \sigma(z) = \frac{1}{1+e^{-z}}$$

Multiclase

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Advertencia

La activación de salida y la función de pérdida deben ser coherentes. MSE suele usarse en regresión; entropía cruzada suele usarse en clasificación.

Función de pérdida: cómo se mide el error

Riesgo empírico

Entrenar consiste en minimizar el error promedio sobre los datos disponibles. La pérdida L compara la predicción $\hat{y}$ con el valor real y .

MSE

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Objetivo de entrenamiento

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L(f_{\theta}(x_i), y_i)$$

Entropía cruzada

$$CE = - \sum_{k=1}^K y_k \log(\hat{p}_k)$$

Lectura estadística

MSE se vincula con errores gaussianos; entropía cruzada con máxima verosimilitud para Bernoulli/categórica. La pérdida codifica supuestos sobre el ruido.

MSE: interpretación geométrica y matemática

Para una observación

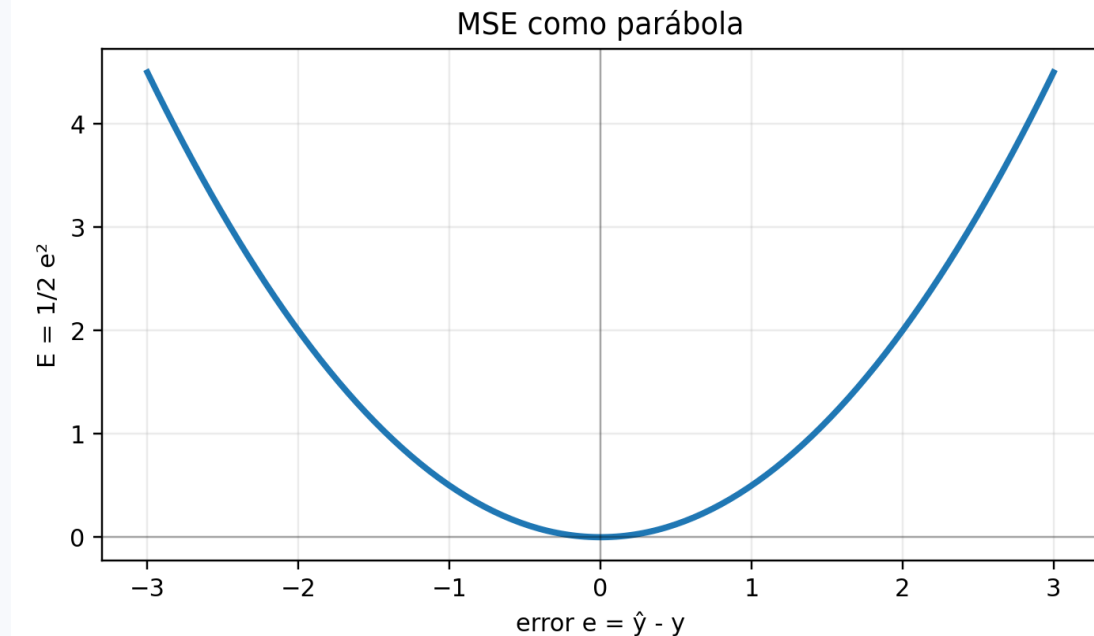
$$E = \frac{1}{2}(y - \hat{y})^2$$

Derivada respecto a $\hat{y}$

$$\frac{\partial E}{\partial \hat{y}} = \hat{y} - y$$

Por qué cuadrático

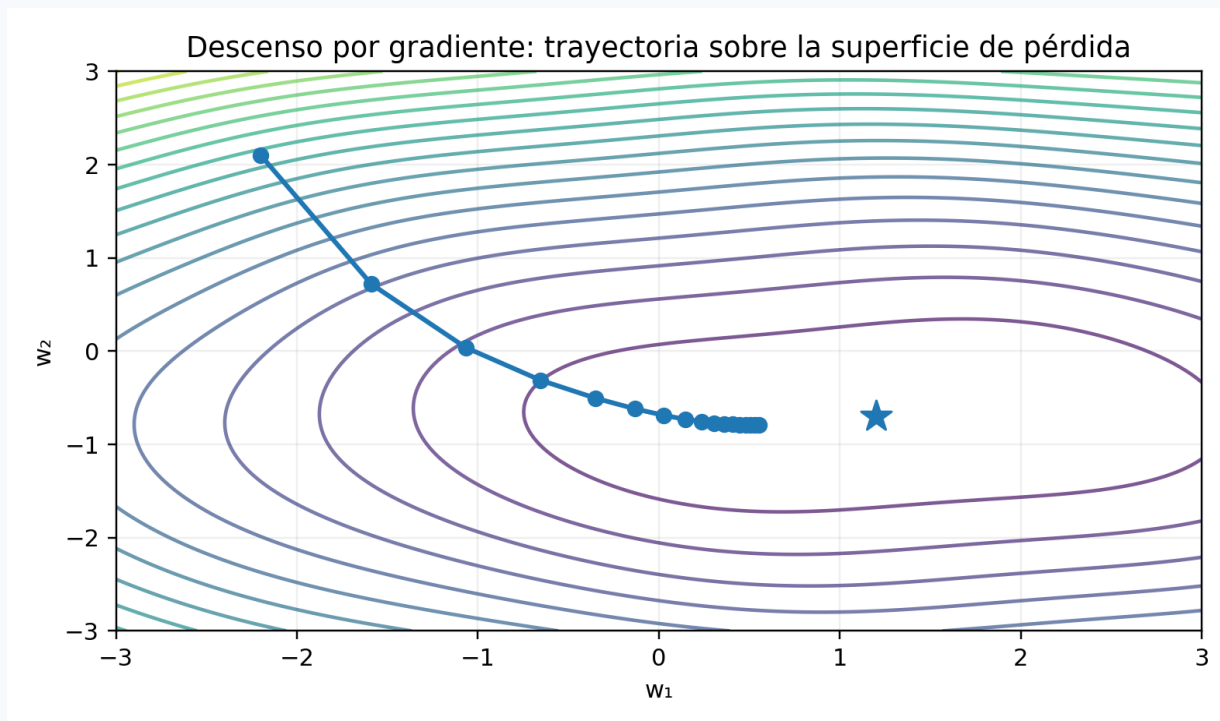
El error al cuadrado penaliza mucho los errores grandes, es diferenciable y genera una superficie suave en muchos problemas.



Cuidado con outliers

Como eleva al cuadrado, MSE puede ser dominado por pocos errores extremos. En algunos contextos conviene MAE, Huber o pérdidas robustas.

Optimización: aprender = minimizar una superficie de pérdida



Parámetros

θ contiene miles, millones o billones de valores. Cada punto θ define una red distinta y un valor de pérdida $J(\theta)$.

Problema

Encontrar θ que reduzca J no es resolver una ecuación cerrada: se usa optimización iterativa.

Descenso básico

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

El gradiente indica cómo cambiarían los errores ante cambios infinitesimales de los parámetros.

Revisión: derivadas parciales

Definición

La derivada parcial mide la tasa de cambio de una función multivariable respecto de una variable, manteniendo las demás constantes.

Definición formal

$$\frac{\partial f}{\partial x_j}(x) = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_j + h, \dots, x_d) - f(x)}{h}$$

Ejemplo

Si $f(w, b) = (y - (wx + b))^2$, entonces podemos preguntar: ¿cómo cambia el error si modifico w ? ¿y si modifico b ?

Ejemplo de derivadas

$$\frac{\partial f}{\partial w} = -2x(y - wx - b), \quad \frac{\partial f}{\partial b} = -2(y - wx - b)$$

En redes neuronales, hay muchísimas derivadas parciales: una por cada peso y por cada bias.

Gradiente: vector de derivadas parciales

Definición

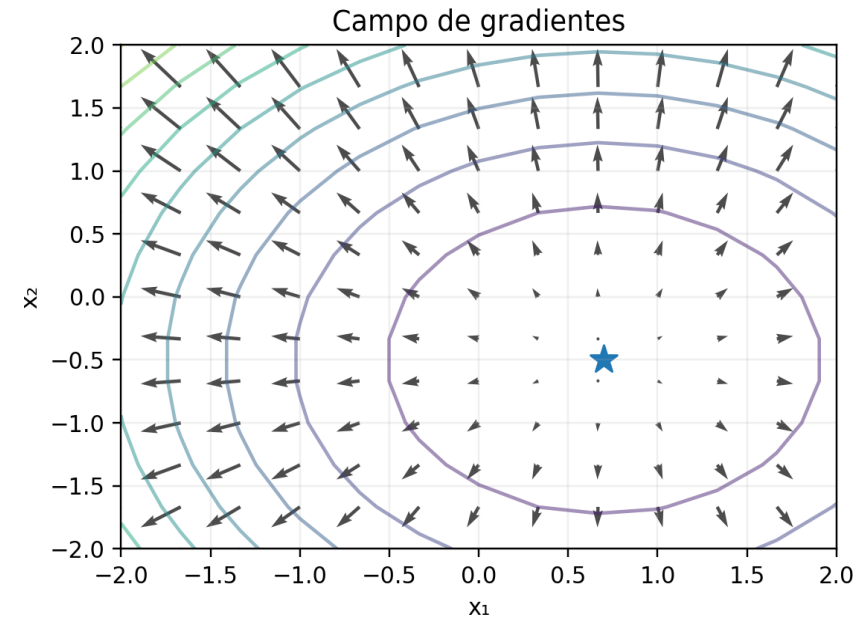
$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_d} \right]^T$$

Interpretación geométrica

El gradiente apunta hacia la dirección de mayor crecimiento local de la función. Por eso, para minimizar, nos movemos en la dirección opuesta.

Dirección de descenso

$$d = -\nabla f(x)$$



En alta dimensión no lo vemos, pero el cálculo es el mismo.

Descenso por gradiente: regla de actualización

Regla principal

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

Tasa de aprendizaje η

η controla el tamaño del paso. Si es muy pequeña, aprende lento; si es muy grande, puede oscilar o divergir.

Mini-batches

En deep learning se estima el gradiente con lotes pequeños. Esto acelera y agrega ruido útil para escapar de regiones pobres.

Algoritmos modernos

SGD, Momentum, RMSProp y Adam son variantes que cambian cómo se usa el gradiente histórico y la escala de cada parámetro.

SGD por mini-batch

$$\nabla J_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla L_i(\theta)$$

Lectura práctica

El entrenamiento es una sucesión de pequeñas correcciones: predigo, mido error, calculo gradiente, actualizo.

Teorema: descenso bajo suavidad L

Enunciado

Si J es diferenciable y su gradiente es L -Lipschitz, entonces para $\eta \leq 1/L$ el paso de gradiente garantiza disminución local de la función.

Garantía de descenso

$$J(\theta - \eta \nabla J(\theta)) \leq J(\theta) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla J(\theta)\|^2$$

Interpretación

El gradiente descendente no es una heurística sin fundamento: bajo condiciones matemáticas, cada paso reduce la pérdida. En redes profundas no siempre hay convexidad, pero la intuición local sigue siendo central.

Caveat profundo

Las redes profundas tienen superficies no convexas. Puede haber puntos silla, valles planos y múltiples soluciones con desempeño similar.

Regla de la cadena: base de backpropagation

Teorema: regla de la cadena multivariable

Si una función se compone de otras funciones diferenciables, la derivada total se obtiene multiplicando derivadas locales a lo largo de la composición.

Caso de red

$$\frac{\partial J}{\partial W^{(\ell)}} = \frac{\partial J}{\partial a^{(L)}} \cdot \dots \cdot \frac{\partial a^{(\ell)}}{\partial W^{(\ell)}}$$

Caso escalar

$$\frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$$

Idea de backprop

No recalcula derivadas desde cero. Propaga “responsabilidades de error” desde la salida hacia atrás, reutilizando cálculos.

Por qué cambió la historia

Sin backprop, entrenar muchas capas era computacionalmente impracticable. Con backprop, el costo es comparable a unas pocas pasadas forward.

Backpropagation en una capa densa

Backpropagation

Error local de salida

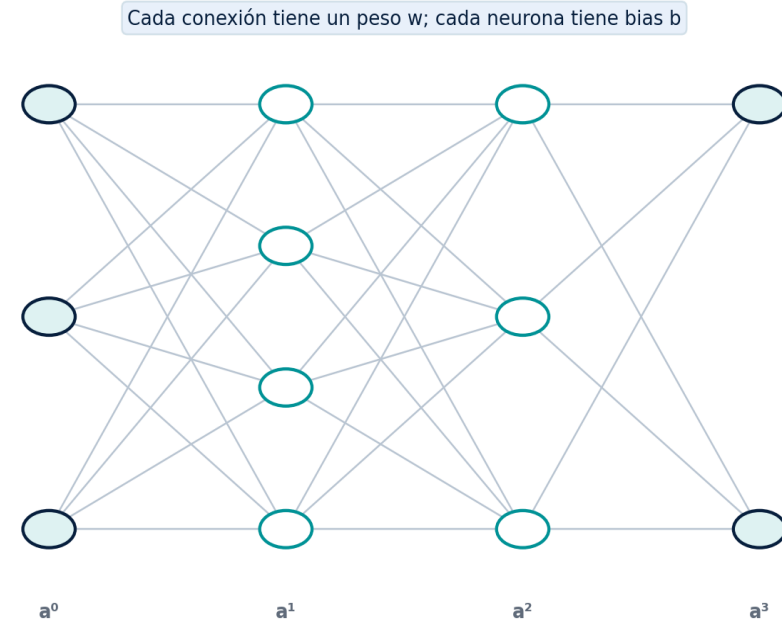
$$\delta^{(L)} = \nabla_{a^{(L)}} J \odot \phi'(z^{(L)})$$

Propagación hacia atrás

$$\delta^{(\ell)} = ((W^{(\ell+1)})^T \delta^{(\ell+1)}) \odot \phi'(z^{(\ell)})$$

Gradientes

$$\frac{\partial J}{\partial W^{(\ell)}} = \delta^{(\ell)} (a^{(\ell-1)})^T, \quad \frac{\partial J}{\partial b^{(\ell)}} = \delta^{(\ell)}$$



Backprop va de derecha a izquierda; forward de izquierda a derecha.

El gradiente del bias: lectura e interpretación

Resultado clave

Para una capa densa, el gradiente del bias es el error local δ . Esto ocurre porque $\partial z / \partial b = 1$.

Derivación breve

$$z_j = \sum_i W_{ji} a_i + b_j \Rightarrow \frac{\partial z_j}{\partial b_j} = 1 \Rightarrow \frac{\partial J}{\partial b_j} = \delta_j$$

Interpretación

Si δ_j es positivo, aumentar b_j aumentaría el error; el descenso por gradiente reduce b_j . Si δ_j es negativo, el algoritmo aumenta b_j .

Ejemplo concreto

En una neurona que detecta “riesgo alto”, el bias ajusta el umbral global de activación. Si el modelo subestima casos de riesgo, el entrenamiento puede subir ese bias para activar la neurona con menos evidencia.

Diseño de arquitectura: profundidad, ancho y parámetros

Ancho

Más neuronas por capa aumentan capacidad para representar variaciones locales. Riesgo: más parámetros y overfitting.

Profundidad

Más capas permiten componer representaciones jerárquicas. Riesgo: gradientes inestables y mayor dificultad de optimización.

Inicialización

Inicializar mal puede saturar activaciones o explotar/aniquilar gradientes. Xavier/He buscan preservar varianza.

Cantidad de parámetros

$$\#\theta = \sum_{\ell=1}^L (n_{\ell}n_{\ell-1} + n_{\ell})$$

Regla de ingeniería

Partir de una arquitectura simple que funcione, medir error, inspeccionar sesgo/varianza y recién después aumentar complejidad.

Buenas prácticas

Normalización de entradas, activaciones adecuadas, batches razonables, monitoreo de validación, early stopping y regularización cuando la varianza domina.

Regularización y generalización

Regularizar

Agregar restricciones o ruido controlado para que el modelo aprenda patrones estables y no memorice detalles accidentales.

Dropout

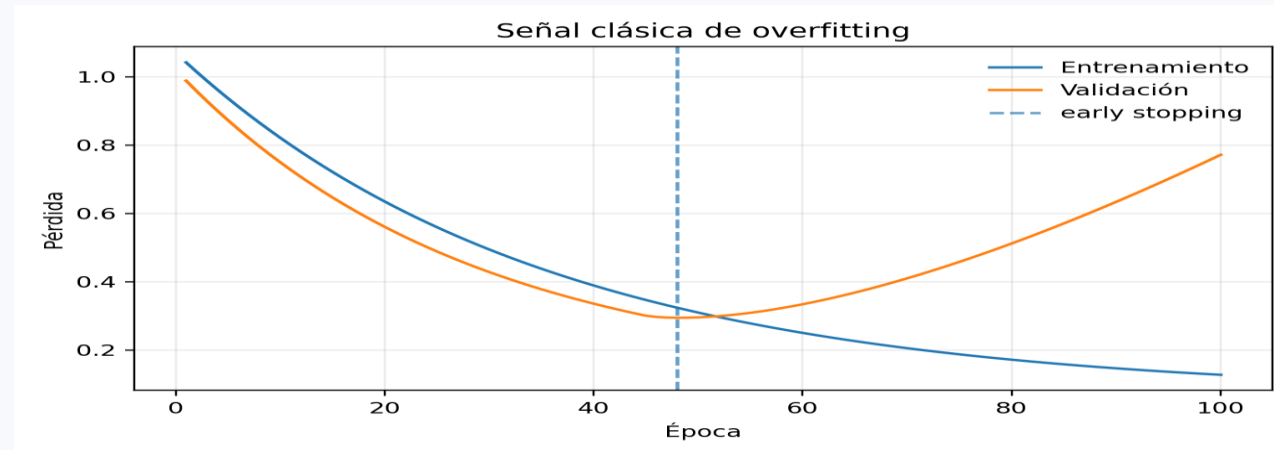
Durante entrenamiento apaga aleatoriamente neuronas. Obliga a distribuir información y reduce coadaptaciones.

L2 / Weight decay

$$J_{reg}(\theta) = J(\theta) + \lambda \|\theta\|_2^2$$

Early stopping

Detener entrenamiento cuando la pérdida de validación deja de mejorar. Es una forma simple y poderosa de regularización.



División de datos: entrenamiento, validación y prueba

Evaluación

Entrenamiento

Datos usados para ajustar pesos y bias.

Validación

Datos usados para elegir hiperparámetros, arquitectura, umbral y early stopping.

Prueba

Datos reservados para estimar desempeño final. No se usan para decidir el modelo.

Train 70%

Valid 15%

Test 15%

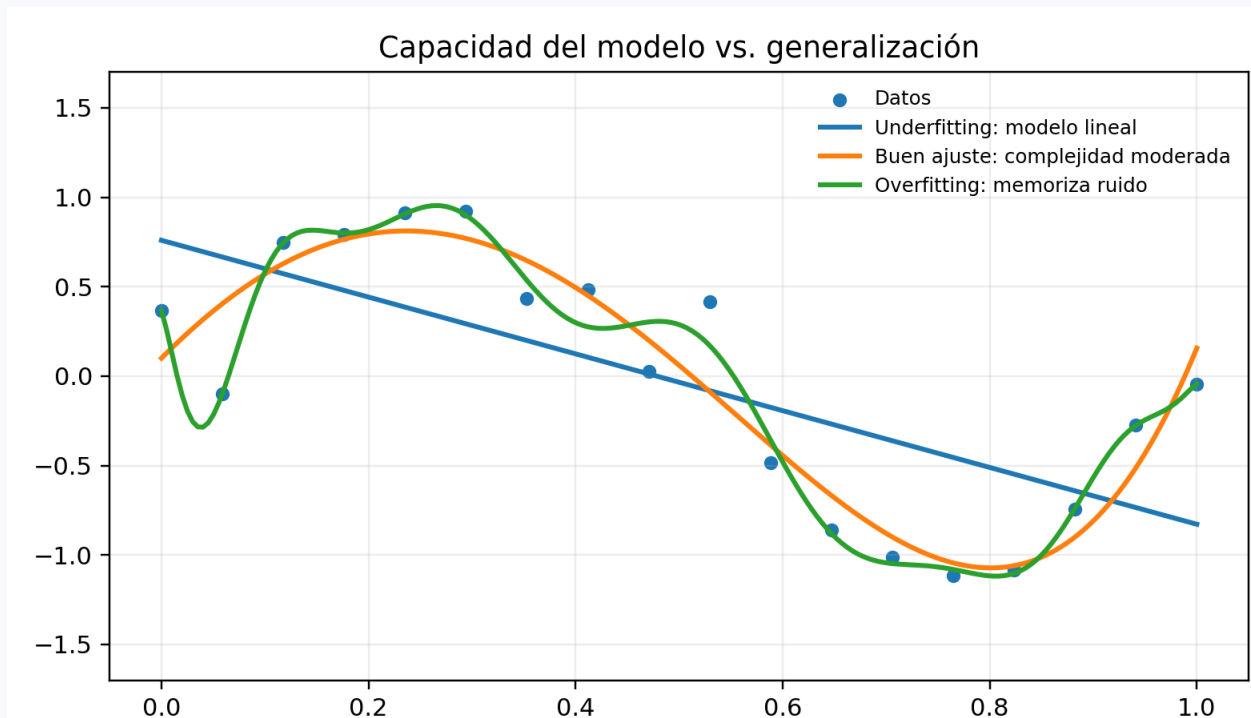
Riesgo crítico: fuga de información

Si el mismo paciente, cliente, sensor o período temporal aparece en train y test, el resultado puede parecer excelente y fallar en producción.

Regla temporal

Cuando el futuro importa, separar por tiempo: entrenar con pasado, validar con período posterior y probar con un período aún más reciente.

Underfitting y overfitting



Underfitting

El modelo es demasiado simple o no entrenó suficiente. Alto error en entrenamiento y validación.

Buen ajuste

Captura regularidades relevantes sin perseguir ruido. Brecha train-validación razonable.

Overfitting

Memoriza particularidades del conjunto de entrenamiento. Bajo error train, alto error validación/test.

Diagnóstico: mirar curvas de aprendizaje, no solo una métrica final.

Teorema: descomposición sesgo-varianza

Descomposición esperada

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \sigma^2 + \text{Bias}(\hat{f})^2 + \text{Var}(\hat{f})$$

Sesgo

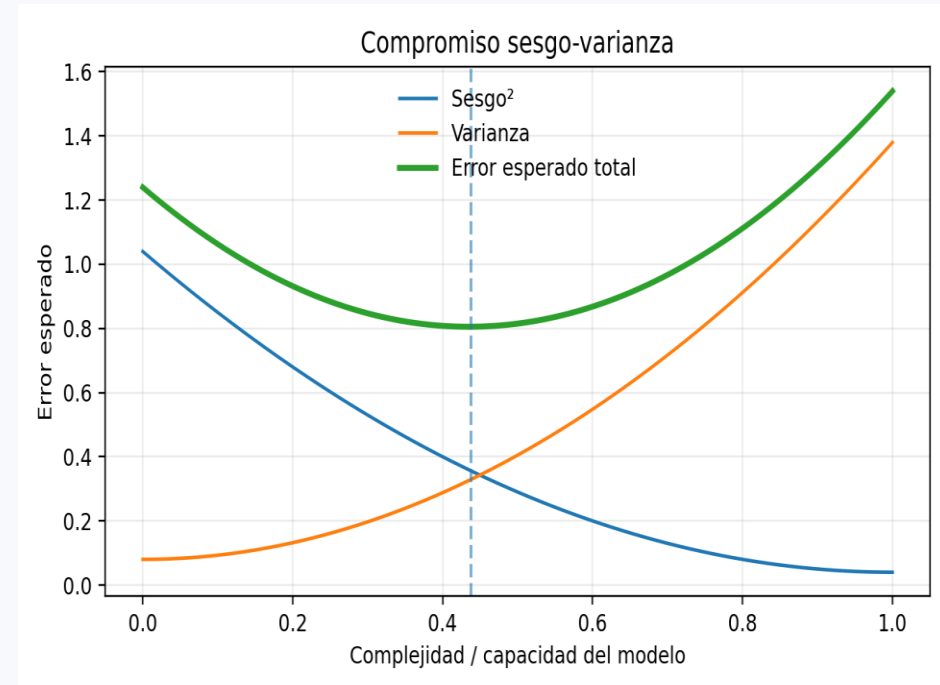
Error sistemático por hipótesis demasiado rígidas: el modelo no puede representar bien el patrón real.

Varianza

Sensibilidad excesiva a la muestra: si cambia el conjunto de entrenamiento, cambia mucho el modelo.

Ruido irreducible

Variación que no puede eliminarse con el modelo porque proviene de mediciones, azar o variables no observadas.



Objetivo: no minimizar solo train; minimizar error esperado sobre datos nuevos.

Apartado: análisis de error

1. Error de datos

Etiquetas incorrectas, mediciones ruidosas, variables omitidas, drift temporal o población de test distinta.

2. Error de modelo

Arquitectura insuficiente, activación inadecuada, mala inicialización o falta de capacidad.

3. Error de optimización

Learning rate mal elegido, pocas épocas, gradientes que explotan/desaparecen o batches inestables.

4. Error de generalización

El modelo funciona en train pero falla en datos nuevos: overfitting, fuga, distribución distinta o sesgo muestral.

5. Error de decisión

La métrica promedio es buena, pero el umbral, costo o impacto por subgrupo es inadecuado.

6. Error humano/proceso

El modelo se usa fuera de contexto, con datos incompletos o sin monitoreo. IA no reemplaza gobernanza.

Checklist MIT para diagnosticar

Separar errores por segmento, clase, período temporal, origen de datos y nivel de confianza. Inspeccionar ejemplos mal clasificados. Comparar contra baseline simple. Revisar si la métrica elegida representa el costo real.

Matriz de confusión: cuando el problema es clasificación

Matriz de confusión

Real 0	86	14
Real 1	9	91
	Pred. 0	Pred. 1

La matriz de confusión depende del umbral de decisión.

Accuracy

$$Acc = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

$$Prec = \frac{TP}{TP+FP}$$

Recall / sensibilidad

$$Recall = \frac{TP}{TP+FN}$$

F1-score

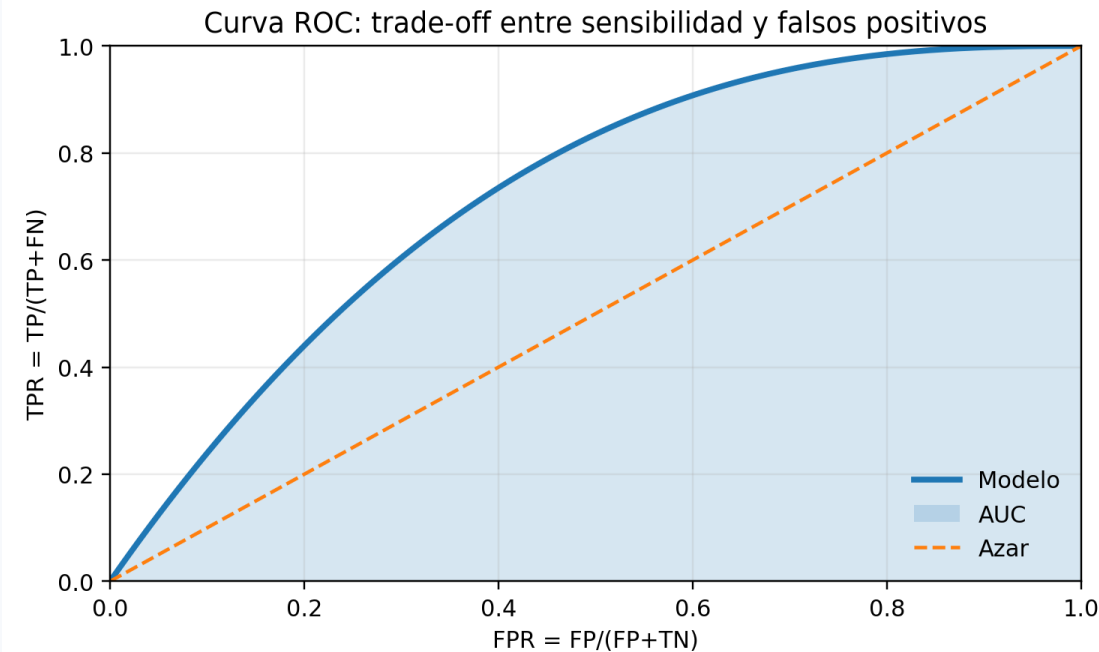
$$F1 = 2 \frac{Prec \cdot Recall}{Prec + Recall}$$

Cuándo usar cada una

Si falsos negativos son costosos, priorizar recall. Si falsos positivos son costosos, priorizar precisión. Con clases desbalanceadas, accuracy puede engañar.

Curva ROC y AUC como integral

Métricas



ROC

La curva ROC evalúa todos los umbrales posibles. En cada umbral se calculan TPR y FPR.

Tasa verdadera positiva

$$TPR = \frac{TP}{TP+FN}$$

Tasa falsa positiva

$$FPR = \frac{FP}{FP+TN}$$

AUC

$$AUC = \int_0^1 TPR(FPR^{-1}(u)) du$$

Advertencia

Con clases muy desbalanceadas, Precision-Recall puede ser más informativa que ROC.

Cross-validation: estimar generalización con menos azar

K-fold cross-validation: cada bloque valida una vez

Val	Train	Train	Train	Train
Train	Val	Train	Train	Train
Train	Train	Val	Train	Train
Train	Train	Train	Val	Train
Train	Train	Train	Train	Val

K-fold

Se divide el dataset en K partes. Cada parte actúa una vez como validación y las K-1 restantes como entrenamiento.

Promedio de desempeño

$$\widehat{M}_{CV} = \frac{1}{K} \sum_{k=1}^K M_k$$

Uso correcto

La validación cruzada ayuda a elegir hiperparámetros. El test final debe seguir separado para estimar desempeño final.

Series temporales

No mezclar futuro con pasado. Usar validación walk-forward o splits temporales.

Ejemplo numérico: red pequeña para regresión

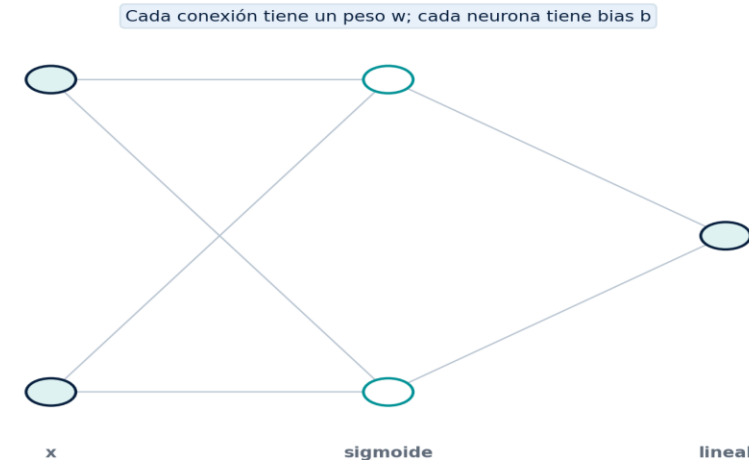
Ejemplo numérico

Problema

Una red con 2 entradas, 1 capa oculta de 2 neuronas sigmoides y salida lineal predice un valor continuo.

Datos

$$x = [1, 2]^T, \quad y = 1.0$$



Pérdida usada: $E = 1/2 (y - \hat{y})^2$

Parámetros iniciales

$$W_1 = \begin{bmatrix} 0.10 & 0.20 \\ -0.10 & 0.10 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0.10 \end{bmatrix}, \quad W_2 = [0.30 \quad -0.20], \quad b_2 = 0.05$$

Ejemplo numérico: forward pass

Ejemplo numérico

Capa oculta

$$z_1 = W_1x + b_1 = \begin{bmatrix} 0.50 \\ 0.20 \end{bmatrix}$$

Activación sigmoide

$$a_1 = \sigma(z_1) = \begin{bmatrix} 0.6225 \\ 0.5498 \end{bmatrix}$$

Salida lineal

$$\hat{y} = W_2a_1 + b_2 = 0.1268$$

Lectura

El modelo predice 0.1268 cuando el valor real es 1.0. La predicción quedó muy baja; el gradiente deberá mover pesos y bias para subir $\hat{y}$.

Pérdida

$$E = \frac{1}{2}(1.0 - 0.1268)^2 = 0.3813$$

Objetivo del update

Cambiar θ para que, ante el mismo x , la próxima predicción se acerque a 1.0.

Ejemplo numérico: backpropagation

Ejemplo numérico

Error de salida

$$\frac{\partial E}{\partial \hat{y}} = \hat{y} - y = -0.8732$$

Gradiente de W_2 y b_2

$$\frac{\partial E}{\partial W_2} = (-0.8732)a_1 = [-0.5435, -0.4801], \quad \frac{\partial E}{\partial b_2} = -0.8732$$

Error oculto

$$\delta_1 = (W_2^T(-0.8732)) \odot \sigma'(z_1) = \begin{bmatrix} -0.0616 \\ 0.0432 \end{bmatrix}$$

Gradiente de W_1

$$\frac{\partial E}{\partial W_1} = \delta_1 x^T = \begin{bmatrix} -0.0616 & -0.1231 \\ 0.0432 & 0.0865 \end{bmatrix}$$

Qué significa

Cada peso recibe una corrección proporcional al error local de su neurona y a la activación que entró por esa conexión.

Ejemplo numérico: actualización con $\eta = 0.1$

Ejemplo numérico

Regla

$$\theta_{new} = \theta - 0.1 \nabla_{\theta} E$$

Nuevos parámetros de salida

$$W_2^{new} = [0.3544, -0.1520], \quad b_2^{new} = 0.1373$$

Nuevos parámetros ocultos

$$W_1^{new} = \begin{bmatrix} 0.1062 & 0.2123 \\ -0.1043 & 0.0914 \end{bmatrix}, \quad b_1^{new} = \begin{bmatrix} 0.0062 \\ 0.0957 \end{bmatrix}$$

Resultado después de una actualización

La nueva predicción pasa de 0.1268 a 0.2784 y la pérdida baja de 0.3813 a 0.2604. Una sola actualización no resuelve el problema, pero se movió en la dirección correcta.

Ejemplo numérico: lectura del bias

Ejemplo numérico

Salida

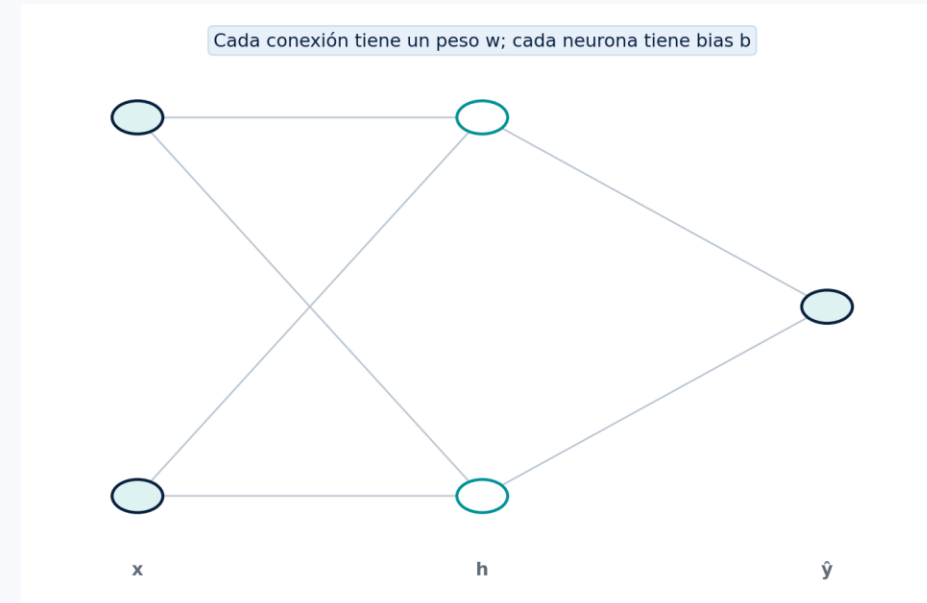
Como $\partial E / \partial b_2 = -0.8732$, el update aumenta b_2 : $0.05 \rightarrow 0.1373$. Esto sube la predicción base porque el modelo estaba subestimando.

Capa oculta

b_1 cambia de forma distinta por neurona: una neurona sube su bias y la otra lo baja. Cada bias ajusta el umbral de activación de su característica aprendida.

Interpretación conceptual

El bias no “mide importancia de una variable”. Mide una tendencia base o umbral de una neurona/capa. Su efecto depende de la activación y del resto de la red.



Bias de salida

$$\hat{y} = W_2 a_1 + b_2$$

Ejemplo real poco usual: audio de colmenas

Caso aplicado

Problema

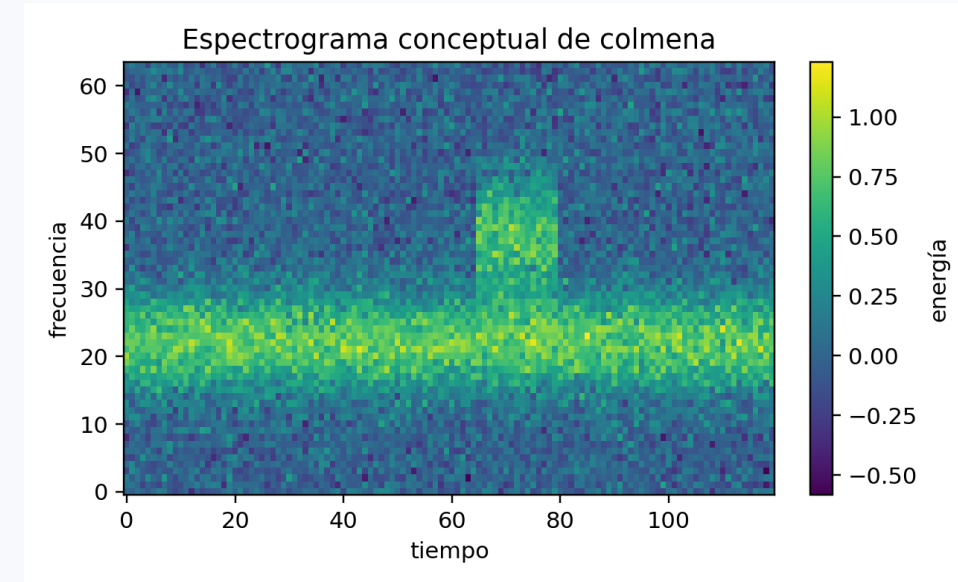
Clasificar si una colmena está en estado normal o bajo estrés usando fragmentos de audio capturados con sensores baratos.

Por qué es buen ejemplo

No es imagen típica ni texto: combina señales acústicas, ruido ambiental, estacionalidad, comportamiento biológico y eventos raros.

Features posibles

Energía por banda, MFCC, frecuencia dominante, entropía espectral, temperatura/humedad y hora del día.



Objetivo: alerta temprana, no diagnóstico definitivo.

División de datos en el caso colmenas

Caso aplicado

Split incorrecto

Cortar audios en fragmentos y mezclar fragmentos de la misma colmena en train y test. El modelo reconoce la colmena, no el estrés.

Split correcto

Separar por colmena, apiario o período temporal. Test debe simular una colmena o temporada no vista.

Validación

Usar validación cruzada agrupada: todos los fragmentos de una colmena quedan en el mismo fold.

Métrica

Si el estrés es raro, accuracy puede ser engañosa. Usar matriz de confusión, recall, precision, F1, ROC/AUC y PR-AUC.

Interpretación

El objetivo no es acertar en promedio, sino detectar eventos relevantes con tasa aceptable de falsas alarmas para el productor.

Análisis de error en el caso colmenas

Caso aplicado

Falsos negativos

Colmenas estresadas clasificadas como normales. Costo alto: se pierde alerta temprana.

Falsos positivos

Colmenas normales marcadas como estrés. Costo: inspecciones innecesarias y pérdida de confianza.

Sesgo de muestra

Si todas las colmenas de train son de una región, el modelo puede fallar en otra por clima, flora o hardware distinto.

Acciones correctivas

Revisar ejemplos mal clasificados, balancear clases, calibrar umbral, agregar variables ambientales, recolectar más estaciones y monitorear drift.

Lectura de red

Una MLP sobre features acústicas puede aprender combinaciones no lineales: por ejemplo, cierto patrón de frecuencia solo importa si ocurre a determinada hora y temperatura.

Umbral ajustable

$$\hat{y} = \mathbb{I}\{p(y = stress|x) > \tau\}$$

Errores comunes al entrenar MLP

Buenas prácticas

Escala de variables

Entrenar con variables en escalas muy distintas dificulta la optimización.
Normalizar o estandarizar.

Learning rate

Demasiado alto: diverge. Demasiado bajo: no aprende en tiempo razonable.

Métrica incorrecta

Optimizar accuracy en clases raras puede ocultar fallas críticas.

Fuga de información

Variables calculadas con futuro, duplicados o misma entidad en train/test inflan resultados.

Capacidad excesiva

Red muy grande para pocos datos: memoriza.

Sin baseline

No comparar con regresión logística, árbol o modelo simple impide saber si la red aporta valor.

Diagnóstico mínimo

Curvas train/validación, matriz de confusión por segmento, análisis de residuos, sensibilidad a hiperparámetros y prueba fuera de distribución cuando sea posible.

Mini-guía para crear una MLP desde cero

1. Definir problema

Regresión, binaria, multiclase o multilabel; decidir métrica de éxito.

2. Preparar datos

Limpieza, normalización, split correcto y prevención de leakage.

3. Elegir arquitectura

Entrada, capas ocultas, activaciones, salida y pérdida coherentes.

4. Entrenar

Forward → loss → backprop → update; monitorear validación.

5. Diagnosticar

Sesgo, varianza, matriz de confusión/ROC/MSE y análisis de error.

6. Iterar

Regularizar, ajustar umbral, recolectar datos o simplificar/aumentar capacidad.

Una red neuronal es un sistema matemático + estadístico + computacional: las tres partes importan.

Cierre: ideas que deben quedar

Cierre

1. Representación

Las MLP son aproximadores universales, pero la utilidad depende del entrenamiento y los datos.

2. Optimización

Gradiente descendente ajusta pesos y bias para reducir la pérdida. Backprop calcula gradientes eficientemente.

3. Bias

El bias no es accesorio: permite umbrales, interceptos y desplazamientos de fronteras de decisión.

4. Generalización

El éxito real está en datos no vistos. Validación, cross-validation y test final son parte del método, no trámites.

5. Error

Analizar error exige mirar sesgo, varianza, ruido, métricas por caso y costos de decisión.

6. Práctica profesional

Un modelo defendible combina matemática, evidencia experimental, interpretación y gobernanza.

Pregunta final: ¿qué evidencia necesitaríamos para confiar en una red neuronal en producción?

Bibliografía recomendada

Cierre

- Goodfellow, I., Bengio, Y. & Courville, A. (2016). Deep Learning. MIT Press.
- Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning. Springer.
- Murphy, K. P. (2022). Probabilistic Machine Learning: An Introduction. MIT Press.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks.
- Nielsen, M. (2015). Neural Networks and Deep Learning.

Para estudiar

Derivar a mano una red 2–2–1 como el ejemplo numérico; luego implementarla en Python/Numpy y comparar gradientes con diferencias finitas.